

Problem Session 2: Process Creation & Pipes

Kiatissak Markmeeshap	67070501005
Tithinan Sobking	67070501018
Worawut Sereethai	67070501040
Chanya Poolketkij	67070501058

Environment

All programs were compiled and run under WSL2 Ubuntu with gcc 15.2.0, using `-std=c11 -D_DEFAULT_SOURCE -Wall -Wextra -O0 -g`. All output shown is real captured output.

1 `fork()` with and without `wait()`

Source

```
1  /*
2  * PS2 item 1.1 -- the fork() "hello world" example, transcribed verbatim from
3  * the Lecture 2 slide "fork(): Hello World".
4  *
5  * The parent does NOT wait for the child, so after fork() returns there are two
6  * runnable processes and the scheduler decides which one prints first. The
7  * order of the last two lines is therefore not deterministic.
8  */
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <unistd.h>
12
13 int
14 main(int argc, char *argv[])
15 {
16     printf("hello world (pid:%d)\n", (int) getpid());
17     int rc = fork();
18     if (rc < 0) {
19         // fork failed; exit
20         fprintf(stderr, "fork failed\n");
21         exit(1);
22     } else if (rc == 0) {
23         // child (new process)
24         printf("hello, I am child (pid:%d)\n", (int) getpid());
25     } else {
26         // parent goes down this path (original process)
27         printf("hello, I am parent of %d (pid:%d)\n",
28                rc, (int) getpid());
29     }
30     return 0;
31 }
```

Listing 1: Source: `src/1_1_fork_hello.c`

```
1  /*
2  * PS2 item 1.2 -- the same program as 1.1, with the parent's branch modified as
3  * shown on the Lecture 2 slide "wait(): Example Usage".
4  *
5  * wait(NULL) suspends the parent until the child terminates, so the child's
6  * line is now guaranteed to be printed before the parent's. <sys/wait.h> is
7  * added because wait() is declared there; everything else is as on the slide.

```

```

8  */
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <unistd.h>
12 #include <sys/wait.h>
13
14 int
15 main(int argc, char *argv[])
16 {
17     printf("hello world (pid:%d)\n", (int) getpid());
18     int rc = fork();
19     if (rc < 0) {
20         // fork failed; exit
21         fprintf(stderr, "fork failed\n");
22         exit(1);
23     } else if (rc == 0) {
24         // child (new process)
25         printf("hello, I am child (pid:%d)\n", (int) getpid());
26     } else {
27         // parent goes down this path (original process)
28         int wc = wait(NULL);
29         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
30                rc, wc, (int) getpid());
31     }
32     return 0;
33 }

```

Listing 2: Source: src/1_2_fork_wait.c

How it works

Both programs are transcribed from the Lecture 2 slides. Each prints a line, calls `fork()`, and then takes a different branch in the parent and in the child. 1.2 differs from 1.1 in one place only: its parent calls `wait(NULL)` before printing, and reports what that call returned as `wc`.

Results

```

$ ./bin/1_1_fork_hello
hello world (pid:679)
hello, I am parent of 680 (pid:679)
hello, I am child (pid:680)

```

Listing 3: `fork()` without `wait()`

```

$ ./bin/1_2_fork_wait
hello world (pid:681)
hello, I am child (pid:682)
hello, I am parent of 682 (wc:682) (pid:681)

```

Listing 4: `fork()` with `wait()`

Which process printed the second line, over 500 runs of 1.1 in each condition

idle machine	parent first: 499	child first: 1
(starting 8 busy loops to contend for the CPU)		
machine under load	parent first: 456	child first: 44
pinned to one CPU	parent first: 500	child first: 0

Listing 5: Which process printed first, counted over repeated runs of 1.1

Discussion

The two outputs differ in the order of the last two lines: in 1.1 the parent printed first, in 1.2 the child did.

In 1.1 the parent does not wait. Once `fork()` returns there are two runnable processes, and nothing in the program orders them — which one reaches its `printf` first is decided by the operating system's scheduler. A single run cannot show that, since it only ever shows one of the possible orders, so we ran 1.1 repeatedly and counted. The child does come first sometimes, but rarely: once in 500 runs on an idle machine. Loading the machine with eight busy loops raised it to 44 in 500. The order really is undecided — it is simply that on an idle multi-core machine the parent almost always wins the race, and making the processes compete for the CPU is what lets the child win often enough to see.

In 1.2 the parent calls `wait(NULL)`, which blocks it until the child terminates. The parent therefore cannot reach its `printf` until the child has already run and exited, so the child's line always comes first — under load or not. Adding `wait()` turns the ordering from an accident of scheduling into a property the program guarantees.

2 Zombies, orphans, and ps

Source

```
1  /*
2  * PS2 item 2.1 -- child is dead while the parent is still running.
3  *
4  * The child exits immediately. The parent never calls wait(), so the kernel
5  * cannot release the child's process table entry: it keeps the exit status
6  * around for a parent that never asks for it. During the parent's sleep(),
7  * 'ps -ef' shows the child as <defunct>.
8  *
9  * The child prints its own getpid() before exiting -- that is the answer to
10 * "report the child's parent process ID" for this scenario.
11 */
12 #include <stdio.h>
13 #include <stdlib.h>
14 #include <unistd.h>
15
16 #define PARENT_LINGER_SECONDS 10
17
18 int main(void)
19 {
20     setvbuf(stdout, NULL, _IOLBF, 0); /* line-buffered, so redirection cannot reorder */
21
22     printf("[parent] pid=%d ppid=%d\n", (int) getpid(), (int) getppid());
23
24     int rc = fork();
25     if (rc < 0) {
26         perror("fork");
27         exit(1);
28     }
29
30     if (rc == 0) {
31         printf("[child ] pid=%d ppid=%d -- exiting immediately\n",
32             (int) getpid(), (int) getppid());
33         exit(0);
34     }
35
36     printf("[parent] forked child pid=%d; sleeping %ds WITHOUT calling wait()\n",
37         rc, PARENT_LINGER_SECONDS);
38     printf("[parent] run 'ps -ef' now: child %d should appear as <defunct>\n", rc);
39     sleep(PARENT_LINGER_SECONDS);
40     printf("[parent] waking up and exiting; the zombie is now reaped by init\n");
41     return 0;
42 }
```

Listing 6: Source: `src/2_1_zombie.c`

```

1  /*
2  * PS2 item 2.2 -- parent is dead while the child is still running.
3  *
4  * The parent lingers just long enough for one 'ps -ef' snapshot to catch both
5  * processes alive, then exits. The child outlives it and becomes an orphan, so
6  * the kernel re-parents it. The child prints getppid() twice -- once while the
7  * parent is alive and once after it has died -- and those two values are the
8  * answer to "report the child's parent process ID" for this scenario.
9  *
10 * Note: an orphan is NOT a zombie. It is alive and running. It only becomes a
11 * defunct entry when it itself exits, and its new parent reaps it immediately,
12 * so 'ps' will not catch this child as <defunct>.
13 */
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <unistd.h>
17
18 #define PARENT_LINGER_SECONDS 2 /* long enough for one snapshot with both alive */
19 #define CHILD_FIRST_WAIT 4 /* child re-checks its parent after this */
20 #define CHILD_SECOND_WAIT 4 /* child stays visible to 'ps' for this long */
21
22 int main(void)
23 {
24     setvbuf(stdout, NULL, _IOLBF, 0);
25
26     printf("[parent] pid=%d ppid=%d\n", (int) getpid(), (int) getppid());
27
28     int rc = fork();
29     if (rc < 0) {
30         perror("fork");
31         exit(1);
32     }
33
34     if (rc == 0) {
35         printf("[child ] pid=%d ppid=%d (parent is still alive)\n",
36             (int) getpid(), (int) getppid());
37         sleep(CHILD_FIRST_WAIT);
38         printf("[child ] pid=%d ppid=%d (parent has died -- re-parented)\n",
39             (int) getpid(), (int) getppid());
40         sleep(CHILD_SECOND_WAIT);
41         printf("[child ] pid=%d exiting\n", (int) getpid());
42         exit(0);
43     }
44
45     printf("[parent] forked child pid=%d; exiting in %ds without calling wait()\n",
46         rc, PARENT_LINGER_SECONDS);
47     sleep(PARENT_LINGER_SECONDS);
48     return 0;
49 }

```

Listing 7: Source: src/2_2_orphan.c

How it works

In 2.1 the child exits immediately while the parent sleeps without ever calling `wait()`. In 2.2 the reverse: the parent exits first while the child keeps running. Each child prints `getppid()` at the moments that matter — in 2.2 both before and after its parent dies. Because both states are transient, each program is run in the background and `ps -ef` is captured on a timer rather than typed by hand.

Results

```

===== system context =====
PID 1 on this system:
  PID  PPID COMMAND
    1     0 systemd

```

```

shell running this script: pid=436

===== 2.1 child is DEAD while the parent is still RUNNING =====
[parent] pid=439 ppid=436
[parent] forked child pid=441; sleeping 10s WITHOUT calling wait()
[parent] run 'ps -ef' now: child 441 should appear as <defunct>
[child ] pid=441 ppid=439 -- exiting immediately

--- ps -ef at t=3s (parent sleeping, child already exited) ---
UID      PID    PPID  C STIME TTY          TIME CMD
kiatisa+  439     436  0 20:44 pts/0    00:00:00 ./bin/2_1_zombie
kiatisa+  441     439  0 20:44 pts/0    00:00:00 [2_1_zombie] <defunct>

--- same rows, PPID made explicit ---
      PID    PPID STAT COMMAND
      439     436  S    2_1_zombie
      441     439  Z    2_1_zombie
[parent] waking up and exiting; the zombie is now reaped by init

--- ps -ef after the parent exited ---
(nothing left: init reaped the zombie when the parent died)

===== 2.2 parent is DEAD while the child is still RUNNING =====
[parent] pid=455 ppid=436
[parent] forked child pid=457; exiting in 2s without calling wait()
[child ] pid=457 ppid=455 (parent is still alive)

--- ps -ef at t=1s (parent and child both alive) ---
UID      PID    PPID  C STIME TTY          TIME CMD
kiatisa+  455     436  0 20:44 pts/0    00:00:00 ./bin/2_2_orphan
kiatisa+  457     455  0 20:44 pts/0    00:00:00 ./bin/2_2_orphan
[child ] pid=457 ppid=301 (parent has died -- re-parented)

--- who adopted the orphan? ---
orphan pid=457 new ppid=301 comm=Relay(303)
for comparison, PID 1 is: systemd
=> the adopter is NOT PID 1: WSL's per-session Relay process is a child subreaper

--- ps -ef at t=5s (parent gone, child re-parented) ---
UID      PID    PPID  C STIME TTY          TIME CMD
kiatisa+  457     301  0 20:44 pts/0    00:00:00 ./bin/2_2_orphan

--- same rows, PPID made explicit ---
      PID    PPID STAT COMMAND
      457     301  S    2_2_orphan
[child ] pid=457 exiting

--- ps -ef after the child finally exited ---
(nothing left: the new parent reaped it immediately)

```

Listing 8: Both scenarios, with `ps` evidence captured on a timer

Discussion

2.1 — the child dies first. The child exited immediately, yet `ps` still listed it as `[2_1_zombie] <defunct>`, state `Z`, with PPID 439 — the parent's PID. A zombie has already given up its memory, its open files and its CPU time; all that remains is its entry in the process table, holding its exit status until the parent collects it.

The cause is the missing `wait()`. The kernel cannot discard the entry before then, or the parent could never learn how its child ended. Each zombie is tiny, but the process table is finite, so a program that forks repeatedly and never reaps eventually cannot create processes at all — the same limit item 3 runs into.

2.2 — the parent dies first. The child reported PPID 455 while its parent lived and PPID 301 afterwards: the kernel re-parented it, because every process must have a parent to report to.

The adopter was `Relay(303)`, not `PID 1` — even though `PID 1` here really is `systemd`. A process can mark itself a *child subreaper*, and orphans are then handed to the nearest such ancestor instead of travelling all the way up; WSL runs one `Relay` per session and it claims that role. The mechanism is the one the lecture describes; only the `PID` differs.

No `<defunct>` entry appeared here, because an orphan is not a zombie: it is alive and merely has a new parent. When it finally exits, that parent reaps it at once — which is what a subreaper is for.

Comparing the two. Only 2.1 leaks anything. An orphan costs nothing; a zombie persists for as long as its negligent parent lives. The problem in 2.1 is not that the child died, but that nobody was listening when it did.

3 How many processes can we fork?

Source

```
1  /*
2   * PS2 item 3 -- how many processes can we fork?
3   *
4   * Structure follows the Hint in the problem sheet: a recursive function where
5   * the parent wait(s) for its child and the child forks the next one. At any
6   * instant every ancestor is blocked in wait() and only the deepest process is
7   * running, so the process count grows by one at a time (n) rather than by
8   * doubling (2^n). That is what keeps this from behaving like a fork bomb.
9   *
10  * No artificial limit is imposed: the program forks until fork() really fails.
11  *
12  * The count is also written to the file named by $FORK_COUNT_FILE, overwritten
13  * on every successful fork. Since the chain only ever goes deeper, that file
14  * ends up holding the highest count reached. This is needed because the
15  * limiting resource may turn out to be memory, in which case the deepest
16  * process is killed outright rather than getting an error back from fork(), and
17  * would otherwise never get to report anything.
18  */
19 #include <stdio.h>
20 #include <stdlib.h>
21 #include <string.h>
22 #include <errno.h>
23 #include <unistd.h>
24 #include <fcntl.h>
25 #include <sys/wait.h>
26
27 static const char *progress_path = NULL;
28
29 static void record(int successful_forks)
30 {
31     char buf[32];
32     int fd, n;
33
34     if (progress_path == NULL) {
35         return;
36     }
37     fd = open(progress_path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
38     if (fd == -1) {
39         return;
40     }
41     n = snprintf(buf, sizeof buf, "%d\n", successful_forks);
42     if (write(fd, buf, (size_t) n) == -1) {
43         /* best effort only */
44     }
45     close(fd);
46 }
47
48 /* If fork() ever does fail, the deepest process reports it and the chain unwinds. */
49 static void fork_deeper(int successful_forks)
50 {
```

```

51     pid_t rc = fork();
52
53     if (rc < 0) {
54         printf("fork() failed: %s (errno=%d)\n", strerror(errno), errno);
55         printf("successful fork() calls: %d\n", successful_forks);
56         return;
57     }
58
59     if (rc == 0) {
60         record(successful_forks + 1);
61         fork_deeper(successful_forks + 1);
62         exit(0);
63     }
64
65     wait(NULL);                /* parent blocks until the chain unwinds */
66 }
67
68 int main(void)
69 {
70     setvbuf(stdout, NULL, _IOLBF, 0);
71     progress_path = getenv("FORK_COUNT_FILE");
72     record(0);
73     fork_deeper(0);
74     return 0;
75 }

```

Listing 9: Source: src/3_fork_count.c

How it works

Following the Hint in the problem sheet, a recursive function forks until `fork()` fails: the parent `wait()`s and only the child forks onward, so exactly one process runs at a time and the count grows by one (n) rather than doubling (2^n). No artificial limit is imposed — the program really does fork until it cannot fork any more.

Because the process that would report the total may be killed outright when memory runs out, the count is also written to a file after every successful fork, and read back from there. The program is run three times — on its own, with 200 extra `sleep` processes running, and after killing those again. Each run is guarded by watching free memory and stopping the whole process group before the kernel's own OOM killer has to intervene, and the next run does not begin until the previous chain has gone and its memory has been returned.

Results

```

per-user process limit (ulimit -u): 35595    -- not reached on this machine
no artificial limit: the program forks until it cannot fork any more
memory guard: run is stopped below 800 MB free

===== measurement 1: the program on its own =====
processes already owned by this user: 9    free memory: 8269 MB
successful fork() calls: 4295
stopped by: the memory guard, with free memory down to 796 MB
           (so this count is a floor, not the true ceiling)
run took 314s
settled after 4s: 9 processes, 8150 MB free

===== spawning 200 background 'sleep' processes =====
processes now owned by this user: 209    free memory: 8013 MB

===== measurement 2: with 200 other processes running =====
successful fork() calls: 4226
stopped by: the memory guard, with free memory down to 761 MB
           (so this count is a floor, not the true ceiling)
run took 306s
settled after 360s: 209 processes, 8002 MB free

===== killing the 200 background processes =====

```

```

processes now owned by this user: 9   free memory: 8276 MB

===== measurement 3: after killing them again =====
successful fork() calls: 4296
stopped by: the memory guard, with free memory down to 787 MB
            (so this count is a floor, not the true ceiling)
run took 315s
settled after 4s: 9 processes, 8277 MB free

```

Listing 10: Three measurements: on its own, with 200 other processes running, and after killing them again

Discussion

Other programs running on the system really do reduce how many times we can fork. On its own the program managed **4295** successful `fork()` calls; with 200 extra `sleep` processes running it managed **4226**; after those were killed it was back to **4296**. Measurements 1 and 3 differ by only 1, so the drop of 69 is well outside the run-to-run variation.

The reason is that whatever runs out is not owned by our program. It is shared across everything the user is running, so processes that have nothing to do with us — shells, editors, those 200 `sleep` processes — consume part of the same budget and leave less for us. Killing them gives it straight back, which is why the third measurement returns to the first.

How much less depends on which resource is the scarce one. Here the drop was 69 rather than 200, because the limit being hit was memory and not a limit on the *number* of processes: the 200 `sleep` processes consumed about 256 MB between them (free memory before each run: 8269 MB, then 8013 MB), and at this depth each of our own processes costs several megabytes, so 256 MB is worth roughly 69 of them. Had the per-user process quota been the binding limit instead, 200 processes would have cost exactly 200 forks, one seat each.

It is worth being clear about what stopped the run, because it was not the process limit — this machine allows 35 595 processes per user and `fork()` never returned an error — nor a shortage of PIDs, of which there are 4 194 304. Memory ran out first, since each process costs the kernel a process table entry, a kernel stack and page tables. The counts above are therefore a floor rather than a true ceiling: each run was stopped once free memory fell below 800 MB, so that the OOM killer was never left to pick a victim of its own.

4 A pipe between parent and child

Source

```

1  /*
2  * PS2 item 4 -- pipe() + fork(), child sends, parent receives.
3  *
4  * pipe() fills fd[0] (read end) and fd[1] (write end). fork() duplicates both
5  * descriptors into the child, so four ends exist afterwards. Each side closes
6  * the end it does not use: the sender closes the read end, the receiver closes
7  * the write end. That is what makes the parent's read() see end-of-file once
8  * the child is gone, instead of blocking forever on a pipe it holds open itself.
9  *
10 * Run with --race to remove the parent's usleep(). Without the usleep the order
11 * of the first two lines is a genuine race between two runnable processes, and
12 * the output will not always match the sheet's expected output.
13 */
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <string.h>
17 #include <unistd.h>
18 #include <sys/wait.h>
19
20 int main(int argc, char *argv[])
21 {

```

```

22  int race = (argc > 1 && strcmp(argv[1], "--race") == 0);
23  int fd[2];
24  char msg[128];
25  char buf[256];
26
27  setvbuf(stdout, NULL, _IOLBF, 0);
28
29  if (pipe(fd) == -1) {
30      perror("pipe");
31      exit(1);
32  }
33
34  pid_t rc = fork();
35  if (rc < 0) {
36      perror("fork");
37      exit(1);
38  }
39
40  if (rc == 0) {
41      /* ---- child: the SENDER, so it closes the read end ---- */
42      close(fd[0]);
43
44      printf("Child: Child PID: %d\n", (int) getpid());
45
46      int n = snprintf(msg, sizeof msg, "Hello from child PID: %d\n", (int) getpid());
47      if (write(fd[1], msg, (size_t) n) == -1) {
48          perror("write");
49          exit(1);
50      }
51
52      close(fd[1]);
53      exit(0);
54  }
55
56  /* ---- parent: the RECEIVER, so it closes the write end ---- */
57  close(fd[1]);
58
59  if (!race) {
60      usleep(50000); /* give the child its turn, to match the expected output */
61  }
62
63  printf("Parent: Parent PID: %d\n", (int) getpid());
64
65  ssize_t n = read(fd[0], buf, sizeof buf - 1);
66  if (n == -1) {
67      perror("read");
68      exit(1);
69  }
70  buf[n] = '\0';
71  printf("Parent: %s", buf);
72
73  close(fd[0]);
74  wait(NULL);
75  return 0;
76 }

```

Listing 11: Source: src/4_pipe.c

How it works

`pipe(fd)` creates one kernel buffer with a read end (`fd[0]`) and a write end (`fd[1]`); `fork()` duplicates both, so each side closes the end it does not use. The child prints its PID, writes its message and exits; the parent prints its own PID, reads the message and prints it.

Results

```
$ ./bin/4_pipe

Child: Child PID: 28136
Parent: Parent PID: 28135
Parent: Hello from child PID: 28136

[exit status: 0]
```

Listing 12: The child sends, the parent receives

Discussion

The output matches the form the problem sheet asks for: the child announced itself, the parent announced itself, and the parent then printed the message the child had sent through the pipe.

`pipe(fd)` produces one buffer inside the kernel with two handles onto it, `fd[0]` to read and `fd[1]` to write, and `fork()` copies the whole descriptor table — so immediately afterwards *four* descriptors refer to that one pipe. The child closes `fd[0]` and the parent closes `fd[1]`, leaving exactly one writer and one reader. That closing is not housekeeping: the kernel signals end-of-file to a reader only when *every* write end has been closed, so a parent still holding its own copy of `fd[1]` would wait forever for an EOF that its own descriptor prevents, and the program would simply hang.

The pipe does guarantee that the third line comes after the first, since the parent's `read()` cannot return until the child has written. It guarantees nothing about the second line: the child's announcement and the parent's are printed by two runnable processes that are not synchronised with each other, so their order is left to the scheduler, exactly as in item 1.1. Our program inserts a brief `usleep()` in the parent purely so the output matches the expected listing, which hides the race rather than resolving it — a fixed delay is a guess about timing, and a wrong one under load. The pipe already provides real synchronisation, and using it, for instance by having the parent print only after its `read()` returns, would order the output without any sleep at all.

5 Pipe edge cases

Source

```
1  /*
2   * PS2 item 5.1 (a) -- the sender tries to read back its own message, having
3   * closed the read end as item 4 requires.
4   *
5   * fd[0] is no longer a valid descriptor in the child, so read() fails
6   * immediately with EBADF. Nothing is consumed from the pipe and the parent
7   * still receives the message normally.
8   */
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <string.h>
12 #include <errno.h>
13 #include <unistd.h>
14 #include <sys/wait.h>
15
16 int main(void)
17 {
18     int fd[2];
19     char msg[128];
20     char buf[256];
21
22     setvbuf(stdout, NULL, _IOLBF, 0);
23
24     if (pipe(fd) == -1) { perror("pipe"); exit(1); }
```

```

25
26 pid_t rc = fork();
27 if (rc < 0) { perror("fork"); exit(1); }
28
29 if (rc == 0) {
30     /* sender, read end properly closed */
31     close(fd[0]);
32
33     int n = snprintf(msg, sizeof msg, "Hello from child PID: %d\n", (int) getpid());
34     write(fd[1], msg, (size_t) n);
35     printf("[child ] wrote %d bytes into the pipe\n", n);
36
37     printf("[child ] now trying to read() back from fd[0], which it closed...\n");
38     ssize_t got = read(fd[0], buf, sizeof buf - 1);
39     if (got == -1) {
40         printf("[child ] read() failed: %s (errno=%d)\n", strerror(errno), errno);
41     } else {
42         buf[got] = '\0';
43         printf("[child ] read() unexpectedly returned %zd bytes: %s", got, buf);
44     }
45
46     close(fd[1]);
47     exit(0);
48 }
49
50 /* receiver */
51 close(fd[1]);
52 usleep(200000); /* let the child finish its attempt first */
53
54 ssize_t got = read(fd[0], buf, sizeof buf - 1);
55 if (got == -1) {
56     perror("[parent] read");
57 } else {
58     buf[got] = '\0';
59     printf("[parent] received %zd bytes: %s", got, buf);
60 }
61
62 close(fd[0]);
63 wait(NULL);
64 return 0;
65 }

```

Listing 13: Source: src/5_1a_read_closed.c

```

1  /*
2   * PS2 item 5.1 (b) -- the sender tries to read back its own message, this time
3   * WITHOUT closing the read end.
4   *
5   * A pipe is not a mailbox addressed to anyone. It is one shared buffer, and
6   * whichever process reads first takes the bytes. Here the child writes and then
7   * immediately reads, so it consumes its own message; the parent gets nothing.
8   *
9   * The parent's read() is protected by alarm(): a SIGALRM handler installed
10  * WITHOUT SA_RESTART makes a blocked read() return EINTR instead of restarting,
11  * so this program can never hang the test run.
12  */
13 #include <stdio.h>
14 #include <stdlib.h>
15 #include <string.h>
16 #include <errno.h>
17 #include <signal.h>
18 #include <unistd.h>
19 #include <sys/wait.h>
20
21 #define PARENT_PATIENCE_SECONDS 5
22
23 static void on_alarm(int sig) { (void) sig; }
24
25 int main(void)
26 {
27     int fd[2];
28     char msg[128];
29     char buf[256];

```

```

30 struct sigaction sa;
31
32 setvbuf(stdout, NULL, _IOLBF, 0);
33
34 if (pipe(fd) == -1) { perror("pipe"); exit(1); }
35
36 pid_t rc = fork();
37 if (rc < 0) { perror("fork"); exit(1); }
38
39 if (rc == 0) {
40     /* sender -- deliberately keeps fd[0] open */
41     int n = snprintf(msg, sizeof msg, "Hello from child PID: %d\n", (int) getpid());
42     write(fd[1], msg, (size_t) n);
43     printf("[child ] wrote %d bytes into the pipe (read end left OPEN)\n", n);
44
45     ssize_t got = read(fd[0], buf, sizeof buf - 1);
46     if (got == -1) {
47         printf("[child ] read() failed: %s\n", strerror(errno));
48     } else {
49         buf[got] = '\0';
50         printf("[child ] read back its OWN message (%zd bytes): %s", got, buf);
51     }
52
53     close(fd[0]);
54     close(fd[1]);
55     exit(0);
56 }
57
58 /* receiver */
59 close(fd[1]);
60
61 memset(&sa, 0, sizeof sa);
62 sa.sa_handler = on_alarm;
63 sigemptyset(&sa.sa_mask);
64 sa.sa_flags = 0; /* no SA_RESTART -> read() returns EINTR */
65 sigaction(SIGALRM, &sa, NULL);
66
67 usleep(300000); /* make sure the child reads first */
68 printf("[parent] now calling read(), giving up after %ds\n", PARENT_PATIENCE_SECONDS);
69 alarm(PARENT_PATIENCE_SECONDS);
70
71 ssize_t got = read(fd[0], buf, sizeof buf - 1);
72 alarm(0);
73
74 if (got == -1 && errno == EINTR) {
75     printf("[parent] read() was still blocked after %ds -- the message is gone\n",
76           PARENT_PATIENCE_SECONDS);
77 } else if (got == -1) {
78     printf("[parent] read() failed: %s\n", strerror(errno));
79 } else if (got == 0) {
80     printf("[parent] read() returned 0 (EOF): all write ends closed, nothing to read\n");
81 } else {
82     buf[got] = '\0';
83     printf("[parent] received %zd bytes: %s", got, buf);
84 }
85
86 close(fd[0]);
87 wait(NULL);
88 return 0;
89 }

```

Listing 14: Source: src/5_1b_read_open.c

```

1  /*
2  * PS2 item 5.2 -- the receiver reads before the sender sends anything.
3  *
4  * read() on a pipe with no data and at least one write end still open BLOCKS.
5  * The parent calls read() at t=0; the child does not write until t=3. The
6  * timestamps printed either side of read() are the evidence: the call does not
7  * return early, and it does not return an error -- it simply parks the process
8  * until data arrives.
9  */
10 #include <stdio.h>

```

```

11 #include <stdlib.h>
12 #include <string.h>
13 #include <time.h>
14 #include <unistd.h>
15 #include <sys/wait.h>
16
17 #define CHILD_DELAY_SECONDS 3
18
19 static double now_seconds(void)
20 {
21     struct timespec ts;
22     clock_gettime(CLOCK_MONOTONIC, &ts);
23     return (double) ts.tv_sec + (double) ts.tv_nsec / 1e9;
24 }
25
26 int main(void)
27 {
28     int fd[2];
29     char msg[128];
30     char buf[256];
31
32     setvbuf(stdout, NULL, _IOLBF, 0);
33
34     if (pipe(fd) == -1) { perror("pipe"); exit(1); }
35
36     double t0 = now_seconds();
37
38     pid_t rc = fork();
39     if (rc < 0) { perror("fork"); exit(1); }
40
41     if (rc == 0) {
42         close(fd[0]);
43         printf("[child ] t=%.3fs sleeping %ds before writing\n",
44               now_seconds() - t0, CHILD_DELAY_SECONDS);
45         sleep(CHILD_DELAY_SECONDS);
46
47         int n = snprintf(msg, sizeof msg, "Hello from child PID: %d\n", (int) getpid());
48         write(fd[1], msg, (size_t) n);
49         printf("[child ] t=%.3fs wrote %d bytes\n", now_seconds() - t0, n);
50
51         close(fd[1]);
52         exit(0);
53     }
54
55     close(fd[1]);
56     printf("[parent] t=%.3fs calling read() -- pipe is still empty\n", now_seconds() - t0);
57
58     ssize_t got = read(fd[0], buf, sizeof buf - 1);
59     double t_return = now_seconds() - t0;
60
61     if (got == -1) {
62         perror("[parent] read");
63     } else {
64         buf[got] = '\0';
65         printf("[parent] t=%.3fs read() returned %zd bytes: %s", t_return, got, buf);
66         printf("[parent] read() was blocked for about %.3f seconds\n", t_return);
67     }
68
69     close(fd[0]);
70     wait(NULL);
71     return 0;
72 }

```

Listing 15: Source: src/5_2_read_first.c

```

1  /*
2   * PS2 item 5.3 -- the sender writes several messages before the receiver reads.
3   *
4   * A pipe is a byte stream, not a queue of messages. The three write() calls do
5   * not create three records; they append bytes to one buffer. A single read()
6   * with a large enough buffer returns all of them at once, and the returned byte
7   * count equals the sum of the three writes -- there is no framing telling the
8   * receiver where one message ended and the next began.

```

```

9  */
10 #include <stdio.h>
11 #include <stdlib.h>
12 #include <string.h>
13 #include <unistd.h>
14 #include <sys/wait.h>
15
16 #define MESSAGE_COUNT 3
17
18 int main(void)
19 {
20     int fd[2];
21     char msg[128];
22     char buf[1024];
23
24     setvbuf(stdout, NULL, _IOLBF, 0);
25
26     if (pipe(fd) == -1) { perror("pipe"); exit(1); }
27
28     pid_t rc = fork();
29     if (rc < 0) { perror("fork"); exit(1); }
30
31     if (rc == 0) {
32         close(fd[0]);
33
34         int total = 0;
35         for (int i = 1; i <= MESSAGE_COUNT; i++) {
36             int n = snprintf(msg, sizeof msg,
37                             "message %d from child PID: %d\n", i, (int) getpid());
38             write(fd[1], msg, (size_t) n);
39             total += n;
40             printf("[child ] write() #%d sent %d bytes\n", i, n);
41         }
42         printf("[child ] %d separate write() calls, %d bytes in total\n",
43               MESSAGE_COUNT, total);
44
45         close(fd[1]);
46         exit(0);
47     }
48
49     close(fd[1]);
50     sleep(1); /* let all three writes land before reading */
51
52     printf("[parent] calling read() ONCE with a %zu-byte buffer\n", sizeof buf);
53     ssize_t got = read(fd[0], buf, sizeof buf - 1);
54     if (got == -1) {
55         perror("[parent] read");
56         exit(1);
57     }
58     buf[got] = '\0';
59
60     printf("[parent] a single read() returned %zd bytes:\n", got);
61     printf("---- begin received data ----\n%s---- end received data ----\n", buf);
62
63     close(fd[0]);
64     wait(NULL);
65     return 0;
66 }

```

Listing 16: Source: src/5_3_multi_message.c

How it works

Four small variants of the item 4 program, each changing one thing. **5.1** is tested twice because the outcome depends on whether the sender still holds the read end: (a) keeps the `close()` from item 4, (b) deliberately leaves it open. In **5.2** the parent reads at $t = 0$ while the child does not write until $t = 3$, with timestamps printed either side of the call. In **5.3** the child sends three messages before the parent performs a single `read()`.

Results

```
$ ./bin/5_1a_read_closed

[child ] wrote 28 bytes into the pipe
[child ] now trying to read() back from fd[0], which it closed...
[child ] read() failed: Bad file descriptor (errno=9)
[parent] received 28 bytes: Hello from child PID: 28171

[exit status: 0]
```

Listing 17: Item 5.1(a) — sender reads with the read end closed.

```
$ ./bin/5_1b_read_open

[child ] wrote 28 bytes into the pipe (read end left OPEN)
[child ] read back its OWN message (28 bytes): Hello from child PID: 28175
[parent] now calling read(), giving up after 5s
[parent] read() returned 0 (EOF): all write ends closed, nothing to read

[exit status: 0]
```

Listing 18: Item 5.1(b) — sender reads with the read end left open.

```
$ ./bin/5_2_read_first

[parent] t=0.000s  calling read() -- pipe is still empty
[child ] t=0.000s  sleeping 3s before writing
[child ] t=3.004s  wrote 28 bytes
[parent] t=3.004s  read() returned 28 bytes: Hello from child PID: 28179
[parent] read() was blocked for about 3.004 seconds

[exit status: 0]
```

Listing 19: Item 5.2 — receiver reads before the sender writes.

```
$ ./bin/5_3_multi_message

[child ] write() #1 sent 32 bytes
[child ] write() #2 sent 32 bytes
[child ] write() #3 sent 32 bytes
[child ] 3 separate write() calls, 96 bytes in total
[parent] calling read() ONCE with a 1024-byte buffer
[parent] a single read() returned 96 bytes:
---- begin received data ----
message 1 from child PID: 28183
message 2 from child PID: 28183
message 3 from child PID: 28183
---- end received data ----

[exit status: 0]
```

Listing 20: Item 5.3 — three writes, one read.

Discussion

5.1 — the sender reads back its own message. The outcome depends entirely on whether the sender still holds the read end, so both cases were tested.

With the read end closed, as item 4 requires, the child’s `read()` failed at once with `EBADF` — “bad file descriptor”. The error is about the *handle*, not the pipe: `fd[0]` is no longer a valid descriptor in that process, so the call never reaches the pipe to discover whether anything is in it. Nothing was consumed, and the parent received the message normally.

With the read end left open, the child read its own 28-byte message back successfully, and the parent then got 0 — end of file — with nothing to show for it. A pipe is not addressed to anybody. It is one shared buffer, and whichever process reads first takes the bytes. The

convention that a pipe runs one way is maintained purely by both sides closing the end they do not use; the kernel does not enforce it. Note the parent's `read()` returned 0 rather than blocking: by then the child had exited, so no write end remained open anywhere, and the kernel reports end-of-file rather than making the reader wait for data that can never arrive.

5.2 — the receiver reads before the sender sends. The parent called `read()` at $t = 0$ on an empty pipe and the call returned at $t = 3.004$ s, immediately after the child wrote. It did not fail and it did not return 0; it blocked, and the process was simply set aside until data arrived.

This is the distinction the previous case turned on. An empty pipe that still has a writer means “no data *yet*”, and the reader waits. An empty pipe with no writers left means “no data *ever*”, and `read()` returns 0. The same empty buffer produces opposite behaviour depending on whether any write end is still open — which is why closing the unused ends correctly matters so much.

5.3 — several messages before a single read. The child made three separate `write()` calls of 32 bytes each. The parent made one `read()` and received 96 bytes: all three messages, concatenated, with nothing marking where one ended and the next began.

A pipe is a byte stream, not a queue of messages. The three writes did not create three records; they appended bytes to one buffer. This is the essential difference between a pipe and a message queue, which preserves message boundaries. If the receiver needs its messages back as separate items, the two sides must agree on a framing convention of their own — a delimiter such as a newline, or a fixed-size length prefix before each message. Here the messages happen to end in newlines, so the receiver could split on those, but that only works because both sides implicitly agreed on it.

In summary. A correct pipe user closes the end it does not need, so that end-of-file is reported when the writers are really gone; expects `read()` to block rather than to fail when data has not arrived yet; and does not assume that one `write()` corresponds to one `read()`.